GESP-7 质量抽检模拟试卷（含答案与解析）

考试时间：60 分钟　满分：100 分　生成时间：2026/05/07 08:39:56

姓名：_____ 班级：_____ 得分：_____

请在答题区域作答；单项选择题、阅读程序题和完善程序题均为单选。除特别说明外，程序均按 C++17 标准理解。

抽样来源：papers/2026 已入库题库文件仓库；蓝图版本：config/blueprint.ts v1；抽样种子：20260507034045

一、单项选择题（共 15 题，每题 2 分，共 30 分）

1. （2 分）GESP-7 算法单选：使用深度优先搜索（DFS）枚举数组 [2, 3, 5, 7, 8] 的所有子集，统计其中元素之和恰好为 15 的子集个数。结果是多少？

A. 4

B. 5

C. 2

D. 3

答案：D

解析：

数组元素为 [2, 3, 5, 7, 8]，需要找出所有子集使得元素之和为 15。通过枚举可以验证，满足条件的子集有：
1. {2, 5, 8} 和为 15
2. {3, 5, 7} 和为 15
3. {7, 8} 和为 15

其他可能的子集如 {2, 3, 10} 中的 10 不在数组中，{15} 本身不在数组中，{2, 3, 5, 5} 包含重复元素（不允许重复使用同一元素）。因此共有 3 个子集。

2. （2 分）给定数组 arr = {8, 3, 5, 2, 9, 1, 7}，对 arr 执行一趟快速排序（以第一个元素为基准，使用 Lomuto 分区方案），请问一趟分区后，基准元素 8 的最终索引是多少？

A. 2

B. 3

C. 4

D. 5

答案：D

解析：

Lomuto 分区过程：初始 i = 0（指向小于区间的末尾），遍历 j 从 1 到 6：
arr[1]=3 ≤ 8 → i=1，交换 arr[1] 与 arr[1]（无变化）；
arr[2]=5 ≤ 8 → i=2，交换 arr[2] 与 arr[2]；
arr[3]=2 ≤ 8 → i=3，交换 arr[3] 与 arr[3]；
arr[4]=9 > 8 → 不动；
arr[5]=1 ≤ 8 → i=4，交换 arr[4]（9）与 arr[5]（1），数组变为 {8,3,5,2,1,9,7}；
arr[6]=7 ≤ 8 → i=5，交换 arr[5]（9）与 arr[6]（7），数组变为 {8,3,5,2,1,7,9}；
最后交换基准元素 arr[0]（8）与 arr[i]（arr[5]=7），数组变为 {7,3,5,2,1,8,9}。
基准 8 的最终索引为 5，对应下标 5（从 0 开始），选项 D 正确。

3. （2 分）GESP-7 算法单选 147：DFS 枚举数组 [2, 3, 5, 7, 8] 的子集，和恰好为 15 的子集个数是多少？（案例线索：median-orbit-restore-147，关注 binary-answer feasibility。）

A. 4

B. 5

C. 2

D. 3

答案：D

解析：

每个元素选或不选，枚举全部子集后，满足和为 15 的共有 3 个。
本题的判定入口是 binary-answer feasibility，不要只按数值外观判断。

4. （2 分）GESP-7 算法单选 332：给定集合 S = {2, 3, 5, 4, 8}，使用深度优先搜索（DFS）枚举所有子集（包括空集）。请问有多少个子集的元素之和恰好等于 15？

A. 2
B. 3
C. 4
D. 1

答案：A

解析：

使用DFS系统枚举所有子集并计算和：
- 空集：和为0，不符合。
- 单元素子集：2,3,5,4,8，均不等于15。
- 双元素子集：检查所有组合，没有和为15的。
- 三元素子集：
 - {2,5,8}：2+5+8=15，符合。
 - {3,4,8}：3+4+8=15，符合。
 - 其他组合如{2,3,10}（10不在集合中）、{4,5,6}（6不在集合中）等均不符合。
- 四元素子集：最小和为2+3+4+5=14<15，最大和为3+4+5+8=20，但检查所有组合，没有和恰好为15的（例如{2,3,4,6}中6不在集合中，{2,3,5,5}中5重复）。
- 五元素子集：总和为2+3+5+4+8=22，不等于15。
因此，共有2个子集的和为15。本题考察DFS枚举子集与求和判断的结合，需要系统枚举避免遗漏。

5. （2 分）GESP-7 算法单选 340：有序数组 [6, 9, 12, 15, 18, 21, 24] 中用 lower_bound 查找 24，返回位置按 1 开始计数，结果是多少？要求：解释 lower_bound 的二分查找过程，并说明当目标值存在时的行为。（案例线索：二分查找边界，关注返回位置定义。）

A. 7
B. 6
C. 8
D. 5

答案：A

解析：

lower_bound 返回第一个不小于目标值的位置。
数组索引（0-based）：0:6, 1:9, 2:12, 3:15, 4:18, 5:21, 6:24
目标值 24，二分查找：
- left=0, right=6
- mid=3, arr[3]=15<24, left=4
- mid=5, arr[5]=21<24, left=6
- mid=6, arr[6]=24=24, 符合不小于，继续向左收缩？但 lower_bound 应返回第一个不小于的位置，所以当 arr[mid] >= target 时，right=mid。标准二分：
初始化 left=0, right=7（右开区间）
while left < right:
 mid = left + (right-left)/2
 if arr[mid] < 24: left = mid+1
 else: right = mid
最终 left=6（0-based），即第一个不小于 24 的索引。转换为 1-based 为 7。所以答案是 A。
但原问题说答案错误，可能是计算有误。验证：数组中 24 存在，lower_bound 应返回其位置，0-based 索引 6，1-based 位置 7。所以正确。

6. （2 分）GESP-7 数据结构单选 721：使用队列模拟约瑟夫环问题。有7个人，编号从1开始依次为1,2,3,4,5,6,7。从编号1开始报数，每次报数到3的人出队，直到剩下最后一人。请问最后留下的人的编号是几？

A. 4

B. 5

C. 6

D. 7

答案：B

解析：

使用队列模拟约瑟夫环过程：
1. 初始队列：1,2,3,4,5,6,7
2. 第一轮：1入队→2入队→3出队，队列变为4,5,6,7,1,2
3. 第二轮：4入队→5入队→6出队，队列变为7,1,2,4,5
4. 第三轮：7入队→1入队→2出队，队列变为4,5,7,1
5. 第四轮：4入队→5入队→7出队，队列变为1,4,5
6. 第五轮：1入队→4入队→5出队，队列变为1,4
7. 第六轮：1入队→4入队→1出队，队列变为4
8. 第七轮：4入队→4出队，队列为空

但上述模拟有误，正确过程应为：
1. 初始队列：1,2,3,4,5,6,7
2. 从1开始报数：1→2→3出队，队列：4,5,6,7,1,2
3. 从4开始报数：4→5→6出队，队列：7,1,2,4,5
4. 从7开始报数：7→1→2出队，队列：4,5,7,1
5. 从4开始报数：4→5→7出队，队列：1,4,5
6. 从1开始报数：1→4→5出队，队列：1,4
7. 从1开始报数：1→4→1出队，队列：4

最终剩下编号4，但原题答案有误，经重新验证正确答案应为4。

原题答案错误，正确答案是4，对应选项A。

7. （2 分）对关键字序列 (15, 9, 20, 12, 17, 11) 建立小根堆，堆调整后，堆顶元素是？

A. 9

B. 11

C. 12

D. 15

答案：A

解析：

小根堆堆顶为最小关键字。序列中最小值为 9，调整后 9 位于堆顶。

8. （2 分）对于长度为n的顺序表，在表头插入一个元素的时间复杂度是？

A. O(1)

B. O(log n)

C. O(n)

D. O(n^2)

答案：C

解析：

在顺序表表头插入元素需要将现有所有元素后移一位，平均移动n个元素，因此时间复杂度为O(n)。

9. （2 分）对于如下 C++ 代码，程序输出的结果是？

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> s;
```

```
    int a[] = {1, 3, 2, 5, 4};
    for (int i = 0; i < 5; i++) {
        if (s.empty() || a[i] < s.top()) {
            s.push(a[i]);
        } else {
            while (!s.empty() && a[i] > s.top()) {
                s.pop();
            }
            s.push(a[i]);
        }
    }
    cout << s.size();
    return 0;
}
```

A. 2

B. 3

C. 4

D. 5

答案：A

解析：

逐步模拟栈操作：初始栈空。i=0，a[0]=1，栈空，压入1。栈：[1]。i=1，a[1]=3，3 > s.top()=1，进入else，while循环：3 > 1，弹出1，栈空。压入3。栈：[3]。i=2，a[2]=2，2 < 3，压入2。栈：[3, 2]。i=3，a[3]=5，5 > 2，弹出2，栈变为[3]；5 > 3，弹出3，栈空；压入5。栈：[5]。i=4，a[4]=4，4 < 5，压入4。栈：[5, 4]。最终栈大小为2，对应选项A。

10. （2 分）GESP-7 数据结构单选 209：给定字符串序列 ['aa', 'bb', 'aa', 'c1', 'bb', 'dd']，使用集合（set）统计其中不同字符串的个数，结果是多少？

A. 3

B. 4

C. 5

D. 6

答案：B

解析：

集合set具有自动去重特性。序列中不同字符串为：'aa'、'bb'、'c1'、'dd'，共4个。

11. （2 分）GESP-7 C++ 单选 1106：给定一个初始为空的 vector<int> v，依次执行以下操作：1. v.push_back(11); 2. v.push_back(7); 3. v.insert(v.begin(), 3); 4. v.erase(v.begin() + 1); 5. v.pop_back(); 请问最终 v 中的元素序列是什么？

A. 3

B. 7

C. 3,11

D. 11,3

答案：A

解析：

1. push_back(11): v = {11};
2. push_back(7): v = {11, 7};
3. insert(begin(), 3): v = {3, 11, 7};
4. erase(begin()+1): 删除索引 1 的元素 11，v = {3, 7};
5. pop_back(): 删除末尾元素 7，v = {3}。最终序列只有一个元素 3。

12. （2 分）GESP-7 C++ 单选 498：执行以下代码：

```
int a = 4;
void inc(int& x) { x++; }
inc(a);
```

cout << a + 6;
输出结果是多少?

A. 10
B. 7
C. 11
D. 5

答案：C

解析：

函数 inc 的参数是引用传递，因此会修改实参 a 的值。调用 inc(a) 后，a 从 4 增加到 5。然后计算 a + 6 得到 5 + 6 = 11，输出 11。

13. （2 分）GESP-7 C++ 单选 814：外层 int x=8; 内层代码块声明 int x=18; 离开内层后输出 x，结果是多少?

A. 18
B. 26
C. 8
D. 10

答案：C

解析：

内层变量只在代码块内有效，离开后访问的是外层 x=8。

14. （2 分）GESP-7 数学单选 811：埃氏筛统计不超过 41 的质数个数，结果是多少? （案例线索：forest-packet-count-811，关注 prime-sieve crossing。）

A. 12
B. 14
C. 20
D. 13

答案：D

解析：

从 2 开始筛去倍数，保留下来的质数共有 13 个。
本题的判定入口是 prime-sieve crossing，不要只按数值外观判断。

15. （2 分）GESP-7 数学单选 1133：计算 10^6 mod 17 的结果是多少?

A. 10
B. 9
C. 12
D. 60

答案：B

解析：

逐步乘法并每次取模，最终余数为 9。

二、阅读程序题（共 3 题，每题 5 小题，共 40 分；小题分值见题内标注）

16. （本题 14 分）GESP-7 程序阅读 885（ALG/medium）：阅读下面的 C++17 程序并回答问题。

```
// case prefix-vertex-filter-885; focus binary-answer feasibility; level medium; kp ALG
// OJCode-inspired training note: prefix-vertex-filter-885
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> a = {3, 5, 7, 9, 11, 13, 15, 17};
```

```
    int target = 3;

    int l = 0, r = (int)a.size() - 1;
    int ans = -1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (a[mid] >= target) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

16.1 （3 分）程序运行后输出的下标是多少？（观察点 1：prefix-vertex-filter-885）

A. 1

B. 0

C. -1

D. 0 的干扰项 1

答案：B

16.2 （3 分）当 a[mid] >= target 时，程序下一步如何缩小区间？（观察点 2：prefix-vertex-filter-885）

A. l=mid+1

B. r=mid-1

C. 直接退出

D. 删除 a[mid]

答案：B

16.3 （3 分）ans 的含义是什么？（观察点 3：prefix-vertex-filter-885）

A. 当前候选的第一个不小于 target 的位置

B. target 的出现次数

C. 数组长度

D. 当前最大值

答案：A

16.4 （3 分）这段查找的时间复杂度是多少？（观察点 4：prefix-vertex-filter-885）

A. O(1)

B. O(log n)

C. O(n)

D. O(n^2)

答案：B

16.5 （2 分）数组有序是该算法正确的必要条件吗？（观察点 5：prefix-vertex-filter-885）

A. 是

B. 否

C. 只对奇数长度是

D. 只对 target 为偶数是

答案：A

解析：

1. 程序实现二分查找第一个不小于 target 的元素下标。初始 l=0, r=7, ans=-1。target=3，mid 计算为 l+(r-l)/2 避免溢出。mid=3 时 a[3]=9>=3，ans=3，r=2；mid=1 时 a[1]=5>=3，ans=1，r=0；mid=0 时 a[0]=3>=3，ans=0，r=-1；循环结束输出 ans=0。
2. 代码逻辑：若 a[mid]>=target，记录 mid 为候选，并向左继续查找（r=mid-1）以寻找更小的满足条件的下标。
3. ans 变量用于记录当前找到的第一个不小于 target 的元素下标，初始为 -1 表示未找到。
4. 每次循环区间长度减半，时间复杂度为 O(log n)。
5. 二分查找要求数组有序，否则无法通过比较大小正确缩小区间。

17. （本题 13 分）给定以下C++17程序，当 n 初始化为 5 时，程序会执行一些计算。请根据程序逻辑回答问题。

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 5;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        a[i] = i * i - 4 * i + 5;
    }
    int max_val = a[0];
    int min_val = a[0];
    for (int i = 1; i < n; ++i) {
        if (a[i] > max_val) max_val = a[i];
        if (a[i] < min_val) min_val = a[i];
    }
    int diff = max_val - min_val;
    vector<int> b;
    for (int i = 0; i < n; ++i) {
        if (a[i] == max_val || a[i] == min_val) {
            b.push_back(i);
        }
    }
    cout << diff << endl;
    cout << b.size() << endl;
    for (int i = 0; i < b.size(); ++i) {
        cout << b[i] << (i == b.size() - 1 ? "" : " ");
    }
    return 0;
}
```

17.1 （3 分）程序的第一行输出（即diff的值）是多少?

A. 4

B. 8

C. 12

D. 16

答案：A

17.2 （3 分）程序的第二行输出（即b的大小）是多少?

A. 1

B. 2

C. 3

D. 4

答案：C

17.3 （3 分）程序的第三行输出（即b的元素，以空格分隔）是什么?

A. 0 2 4
B. 0 1 4
C. 1 3 4
D. 0 3 4

答案：A

17.4 （2 分）在程序执行过程中，变量max_val被更新的次数是多少？

A. 0
B. 1
C. 2
D. 3

答案：A

17.5 （2 分）如果 n 初值为 5，程序中数组a的第4个元素（即a[3]）的值是多少？

A. 1
B. 2
C. 3
D. 4

答案：B

解析：

1. 当n=5时，数组a的值为：a[0]=5, a[1]=2, a[2]=1, a[3]=2, a[4]=5。最大值max_val为5，最小值min_val为1，因此diff = 5 - 1 = 4。选项A正确。
2. 根据数组a，最大值5出现在索引0和4，最小值1出现在索引2。因此b向量收集索引0,2,4，大小b.size()=3。选项C正确。

3. b向量中存储的索引是那些a[i]等于最大值或最小值的i。由上一问，b={0,2,4}。输出时，循环从i=0到b.size()-1，输出b[i]，并在非最后一个元素后加空格。所以输出为'0 2 4'。选项A匹配。
4. 初始max_val=a[0]=5。然后循环从i=1到4。当i=1，a[1]=2<5，不更新。i=2，a[2]=1<5，不更新。i=3，a[3]=2<5，不更新。i=4，a[4]=5等于5，但条件是'if (a[i] > max_val)'，严格大于才更新，5不大于5，所以不更新。因此，在循环中，max_val没有被更新（即条件赋值次数为0）。初始赋值不算更新，所以更新次数为0。选项A正确。
5. 数组a索引从0开始，第4个元素是a[3]。根据公式a[i]=i*i-4*i+5，当i=3时，计算：3*3 - 4*3 + 5 = 9 - 12 + 5 = 2。所以a[3]=2。选项B正确。

18. （本题 13 分）阅读以下C++程序，根据程序中的初始值，回答后续问题。

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    int n = 6;
    int arr[] = {3, 1, 4, 2, 5, 1};
    stack<int> s;
    int result = 0;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && s.top() <= arr[i]) {
            s.pop();
        }
        s.push(arr[i]);
        result += s.size();
    }
    cout << "最终栈大小: " << s.size() << endl;
    cout << "栈顶元素: " << s.top() << endl;
    cout << "累积和result: " << result << endl;
    return 0;
}
```

18.1 （3 分）程序最终输出的“最终栈大小”是多少？

A. 1

B. 2

C. 3

D. 4

答案：B

18.2 （3 分）程序最终输出的“栈顶元素”是多少？

A. 1

B. 2

C. 5

D. 9

答案：A

18.3 （3 分）程序最终输出的“累积和result”是多少？

A. 7

B. 8

C. 9

D. 10

答案：C

18.4 （2 分）在循环执行过程中，栈`s`被弹出（pop）操作的总次数是多少？

A. 3

B. 4

C. 5

D. 6

答案：B

18.5 （2 分）循环执行过程中，栈`s`的大小（元素个数）的最大值是多少？

A. 1

B. 2

C. 3

D. 4

答案：B

解析：

1. 逐步模拟程序执行：
- i=0: 栈空，直接push(3)，栈为[3]，result += 1，result=1。
- i=1: 栈顶3 > 1，不弹栈，push(1)，栈为[3,1]，result += 2，result=3。
- i=2: 栈顶1 <= 4，弹出1；栈顶3 <= 4，弹出3；栈空，push(4)，栈为[4]，result += 1，result=4。
- i=3: 栈顶4 > 2，不弹栈，push(2)，栈为[4,2]，result += 2，result=6。
- i=4: 栈顶2 <= 5，弹出2；栈顶4 <= 5，弹出4；栈空，push(5)，栈为[5]，result += 1，result=7。
- i=5: 栈顶5 > 1，不弹栈，push(1)，栈为[5,1]，result += 2，result=9。
2. 最终栈为[5,1]，大小为2，栈顶元素为1，累积和result为9。
3. 弹出操作发生在i=2（弹出1和3，2次）和i=4（弹出2和4，2次），共4次。
4. 栈大小变化：i=0后为1，i=1后为2，i=2后为1，i=3后为2，i=4后为1，i=5后为2，最大值为2。

三、完善程序题（共 2 题，共 30 分；各空分值见题内标注）

19. （本题 15 分，共 4 空）下面程序实现了简单选择排序。请填入正确代码完成排序功能。

```
#include <iostream>
#include <vector>
using namespace std;
```

```
void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < BLANK1; ++i) {
        int minIndex = i;
        for (int j = BLANK2; j < n; ++j) {
            if (BLANK3) minIndex = j;
        }
        if (minIndex != i) BLANK4;
    }
}

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) cin >> arr[i];
    selectionSort(arr);
    for (int i = 0; i < n; ++i) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

样例输入：

```
5
3 1 4 1 5
```

样例输出：

```
1 1 3 4 5
```

空 1（BLANK1，4 分）

A. n - 1

B. n

C. n / 2

D. n + 1

答案：A

空 2（BLANK2，4 分）

A. i + 1

B. i

C. 0

D. i - 1

答案：A

空 3（BLANK3，4 分）

A. arr[j] < arr[minIndex]

B. arr[j] > arr[minIndex]

C. arr[j] == arr[minIndex]

D. arr[j] != arr[minIndex]

答案：A

空 4（BLANK4，3 分）

A. swap(arr[i], arr[minIndex])

B. swap(arr[minIndex], arr[i])

C. arr[i] = arr[minIndex]

D. arr[minIndex] = arr[i]

答案：A

解析：

1. 选择排序外层循环控制排序的趟数，共n-1趟。因为最后一个元素自然有序。
2. 内层循环从i+1开始，寻找未排序部分的最小值。
3. 若找到更小的元素，则更新最小索引。

4. 交换当前元素与最小元素。选项B顺序不同但结果相同，但通常习惯写法为swap(arr[i], arr[minIndex])。由于两个值交换，AD都正确，但A顺序符合常规。这里选择A。

参考完整代码：

```
#include <iostream>
#include <vector>
using namespace std;

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIndex]) minIndex = j;
        }
        if (minIndex != i) swap(arr[i], arr[minIndex]);
    }
}

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) cin >> arr[i];
    selectionSort(arr);
    for (int i = 0; i < n; ++i) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

20. （本题 15 分，共 4 空）以下C++代码实现了一个简易的基于数组的环形队列，其中`front`和`rear`分别指向队首和队尾的下一个位置。队列的最大容量为`capacity`，当前元素个数由`size`变量维护。请补全代码中的空白，以实现入队和出队操作。

```
#include <iostream>
using namespace std;

const int MAXN = 100;

class CircularQueue {
private:
    int data[MAXN];
    int front, rear, size, capacity;
public:
    CircularQueue(int cap) {
        front = 0;
        rear = 0;
        size = 0;
        capacity = cap;
    }

    bool enqueue(int x) {
        if (BLANK1) {
            return false; // 队列已满
        }
        data[rear] = x;
        BLANK2;
        size++;
        return true;
    }

    bool dequeue(int &x) {
        if (BLANK3) {
            return false; // 队列为空
        }
        x = data[front];
        BLANK4;
        size--;
        return true;
```

```
    }
};

int main() {
    CircularQueue q(5);
    int val;
    q.enqueue(10);
    q.enqueue(20);
    if (q.dequeue(val)) {
        // 首次出队，val被赋值
    }
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.enqueue(60); // 此时应满
    if (q.dequeue(val)) {
        // 第二次出队，val被赋值
    }
    q.enqueue(70);
    while (q.dequeue(val)) {
        cout << val << " ";
    }
    return 0;
}
```

样例输入：

样例输出：

```
30 40 50 60 70
```

空 1（BLANK1，4 分）

A. size == capacity

B. front == rear

C. size == MAXN

D. rear == capacity

答案：A

空 2（BLANK2，4 分）

A. rear = (rear + 1) % capacity

B. rear = (rear + 1) % MAXN

C. rear++

D. rear = rear % capacity + 1

答案：A

空 3（BLANK3，4 分）

A. front == rear

B. size == 0

C. front == 0 && rear == 0

D. size != capacity

答案：B

空 4（BLANK4，3 分）

A. front = (front + 1) % capacity

B. front = (front + 1) % MAXN

C. front++

D. front = (front + 1) % size

答案：A

解析：

1. 队列已满的条件是当前元素个数`size`等于最大容量`capacity`。选项B是空队列的条件；选项C使用了全局常量`MAXN`，但实际容量可能小于`MAXN`；选项D未考虑环形结构，且`capacity`可能小于`MAXN`。
2. 环形队列的尾指针移动需要取模，模数应为实际容量`capacity`，以确保在`capacity`范围内循环。选项B使用`MAXN`可

能超出实际容量；选项C未考虑环形；选项D的公式会导致索引错误。
3. 队列为空的条件是当前元素个数`size`为0。选项A在环形队列中`front == rear`可能表示队满或队空，但这里通过`size`变量明确维护状态，更直接可靠；选项C只适用于初始状态；选项D是队未满的条件。
4. 与尾指针移动类似，首指针移动也应使用`capacity`取模以实现环形。选项B使用`MAXN`可能错误；选项C未考虑环形；选项D使用`size`取模会导致逻辑错误。

参考完整代码：

```
#include <iostream>
using namespace std;

const int MAXN = 100;

class CircularQueue {
private:
    int data[MAXN];
    int front, rear, size, capacity;
public:
    CircularQueue(int cap) {
        front = 0;
        rear = 0;
        size = 0;
        capacity = cap;
    }

    bool enqueue(int x) {
        if (size == capacity) {
            return false; // 队列已满
        }
        data[rear] = x;
        rear = (rear + 1) % capacity;
        size++;
        return true;
    }

    bool dequeue(int &x) {
        if (size == 0) {
            return false; // 队列为空
        }
        x = data[front];
        front = (front + 1) % capacity;
        size--;
        return true;
    }
};

int main() {
    CircularQueue q(5);
    int val;
    q.enqueue(10);
    q.enqueue(20);
    if (q.dequeue(val)) {
        // 首次出队，val被赋值
    }
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.enqueue(60); // 此时应满
    if (q.dequeue(val)) {
        // 第二次出队，val被赋值
    }
    q.enqueue(70);
    while (q.dequeue(val)) {
        cout << val << " ";
    }
    return 0;
}
```